InDigital XP

Criterios Generales de Evaluación

- Backend Gestionado por Gradle ó Maven
- Backend Desarrollado en Springboot con Java 17 a más
- Frontend desarrollado React o Angular
- Frontend Gestionado por npm o yarn

InDigital XP [Reto 004] Senior Fullstack Developer 1/2

- Crear microservicio customers con la siguiente funcionalidad:
    - Creación de cliente (nombre, apellido, email, dni, fecha creación, fecha de nacimiento)
    - Consulta de clientes por (dni, email) o sin filtros.
    - Consulta de indicadores:
        - Cantidad de clientes nacidos por mes/año
        - Mes/Año con mayor cantidad de clientes nacidos
        - Mes/Año con menor cantidad de clientes nacidos
        - Taza de Natalidad de cada mes

- Postman collection para consumir los 2 endpoints del microservicio

- Código Subido a Github

- Archivo README.md documentando el funcionamiento e instalación del producto

- Al menos una prueba unitaria para una de las operaciones del servicio

InDigital XP [Reto 004] Senior Fullstack Developer 2/2

- Aplicación WEB Customers con la siguiente funcionalidad:
    - Consulta de clientes en tabla o lista
    - Filtro de búsqueda de clientes por dni ó email
    - Formulario de creación de clientes
    - Consulta de indicadores en tabla ó gráfico
    - Comunicación con el backend previamente creado

- Código Subido a Github

- Archivo README.md documentando el funcionamiento e instalación del producto

- Al menos una prueba unitaria

1. Resumen Ejecutivo & Acceso al Repositorio

Se completó la solución integral para el **Reto 004 - Senior Fullstack Developer**. El proyecto fue diseñado bajo estándares de Clean Architecture, inmutabilidad y pruebas automatizadas, desplegándose de manera unificada y autónoma.

Repositorio GitHub (Público):	https://github.com/JordyMontalvo/reto-tecnico
Swagger UI (Documentación en vivo):	http://localhost:8080/swagger-ui.html
Colección Postman:	Ubicada en postman/Customers_API.postman_collection.json
Docker Compose:	Orquestación lista en docker-compose.yml (Backend + Frontend)

Instrucciones de Ejecución para el Evaluador:

Opción 1: Docker Compose (Recomendado)
git clone https://github.com/JordyMontalvo/reto-tecnico.git
cd reto-tecnico
docker compose up --build
• Frontend: **http://localhost:3000**
• Backend API & Swagger: **http://localhost:8080/swagger-ui.html**

Opción 2: Ejecución Local Nativa
1. Backend (Spring Boot 3.3.5 / Java 21):
cd backend && ./gradlew bootRun (inicia en :8080 con 10 clientes semilla)
2. Frontend (React 19 / TypeScript / Vite):
cd frontend && npm install && npm run dev
• Acceso: **http://localhost:5173** (o 5174 si está ocupado)

Matriz de Cumplimiento de Requerimientos:

Requerimiento	Implementación	Estado
Backend Gradle / Spring Boot / Java 17+	Java 21 LTS + Spring Boot 3.3.5 + Gradle Wrapper 9.x	**100% CUMPLIDO**
CRUD & Filtros por DNI / Email	Endpoints /api/v1/customers con validaciones Jakarta y búsqueda	**100% CUMPLIDO**
Indicadores y Tasa Natalidad	Algoritmos con YearMonth, Max, Min y BigDecimal HALF_UP	**100% CUMPLIDO**
Postman Collection & READMEs	Colección Postman v2.1 + README general y módulos individuales	**100% CUMPLIDO**
Frontend React / NPM / Dashboard	React 19 + TypeScript + Vite + Chart.js + Exportación Excel CSV	**100% CUMPLIDO**
Pruebas Unitarias Automatizadas	22 tests backend (88% JaCoCo, 96% service) + 11 tests frontend Vitest	**100% CUMPLIDO**

2. Desarrollo Backend: Spring Boot 3.3.5 & Swagger OpenAPI 3.0

Arquitectura & Buenas Prácticas:

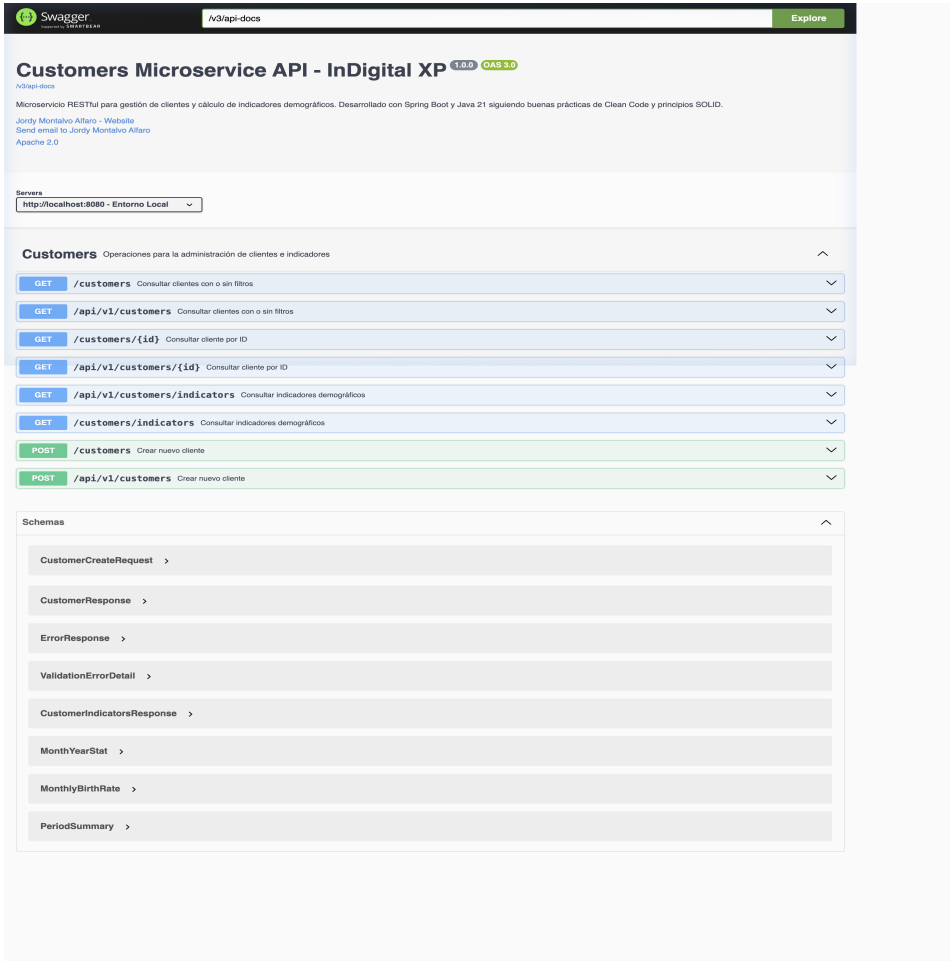
• **Java Records en DTOs:** Inmutabilidad estricta y contratos seguros de datos.
• **Cálculo dinámico de edad:** Basado en Period.between(...) para evitar obsolescencia.
• **Manejo de Errores RFC 7807:** Respuestas estructuradas con 400 Bad Request en validaciones y 409 Conflict ante duplicados de DNI/email.
• **Algoritmo de Tasa de Natalidad:** Agrupación por YearMonth, detección de picos/valles y porcentaje mensual sobre el total calculado con BigDecimal y redondeo HALF_UP.

Endpoints Disponibles:

• POST /api/v1/customers: Registro con validación
• GET /api/v1/customers: Listado con filtros ?dni= y ?email=
• GET /api/v1/customers/indicators: Estadísticas y tasas
• GET /api/v1/customers/{id}: Detalle individual
• GET /actuator/health & /swagger-ui.html

Pruebas Backend (22 tests automatizados):

• CustomerServiceTest: Cobertura de cálculos y casos borde.
• CustomerControllerTest: MockMvc con headers y códigos HTTP.
• CustomerRepositoryTest: Persistencia y unicidad JPA.
• **Cobertura JaCoCo: 88% Global, 96% Capa de Servicio.**



3. Desarrollo Frontend: React 19, TypeScript & Experiencia de Usuario

Funcionalidades de la SPA:

- **Dashboard Demográfico:** 4 KPIs analíticos y gráficos Chart.js interactivos con switch para vista tabla.
- **Búsqueda en Vivo:** Filtro instantáneo por DNI (8 dígitos) y correo con botón de reset.
- **Modal de Registro:** Validaciones reactivas, preview automático de edad calculada y prevención de fechas futuras.
- **Exportación a CSV (Microsoft Excel):** Descarga con BOM UTF-8 (\uFEFF) respetando filtros de búsqueda.
- **Acceso Rápido a Swagger:** Botón en barra superior.
- **11 Pruebas Unitarias con Vitest.**

